Only intuitive explanations can be offered. While uniform vertex distribution reduces clutter, the implied uniform edge lengths leave an undistorted impression of the graph. Since "clutter" and "distortion" already have physical connotations, it seems fairly natural to start thinking of a more specific physical analogy.

We are used to observing even spacing between repelling objects. This makes it natural to imagine vertices behaving like charged balls to satisfy the first criterion. A physical analogy for the second criterion is also easy to find, since it states that we should not allow adjacent balls to drift too far apart. Springs replacing edges will do the job. Springs are better suited than, for example, sticks or ropes, because they can be both extended and compressed to allow moderate distortion, but exert increasingly strong forces when deviating further from their natural length. Moderate distortion may be inevitable, since it is impossible to represent every graph with straight edges of equal length. Note that it is $\mathcal{NP}$-hard to decide whether an arbitrary graph has a straight-line embedding with equal edge lengths in any number of dimensions (Johnson, 1982) or just any planar straight-line embedding (Eades and Wormald, 1990).

Figure 4.1 illustrates the imaginary substitution of vertices and edges with charged balls and connecting springs, respectively. If the system is let go, it attains an equilibrium state in which all forces cancel each other, and the substitution can be reversed to obtain a straight-line drawing that satifies the criteria from above at least approximately.

Formally, such a model can be expressed either in terms of forces acting on the physical objects, or in terms of a potential energy reflecting the internal stress of the system and thus describing how well a configuration matches the design goals modeled in the system. Algorithms to simulate a system's relaxation typically try to move the objects iteratively into stable states in which all forces cancel each other, or to minimize the energy directly. Prominent examples from each kind of formalization are described in the following two sections. In Section 4.4, we give examples for creative use of these analogies in more advanced layout models that include various criteria for good layout.

4.2 Force-Directed Placement

The seminal paper for physical modeling in graph drawing is a short text of Eades (1984), though closely related methods had already been described in the context of VLSI design (Fisk et al., 1967; Quinn and Breuer, 1979). Given a connected undirected graph $G = (V, E)$, let $p = (p_v)_{v \in V}$ be a vector of vertex positions $p_v = (x_v, y_v)$ in the plane. We denote by $\|p_v - p_u\|$ the length of the difference vector $p_v - p_u$, which is the Euclidean distance between positions p_u and p_v. Furthermore, we denote by $\overrightarrow{p_u p_v}$ the unit length vector $\frac{p_v - p_u}{\|p_v - p_u\|}$ pointing from p_u to p_v. The model of Eades (1984), now known as the

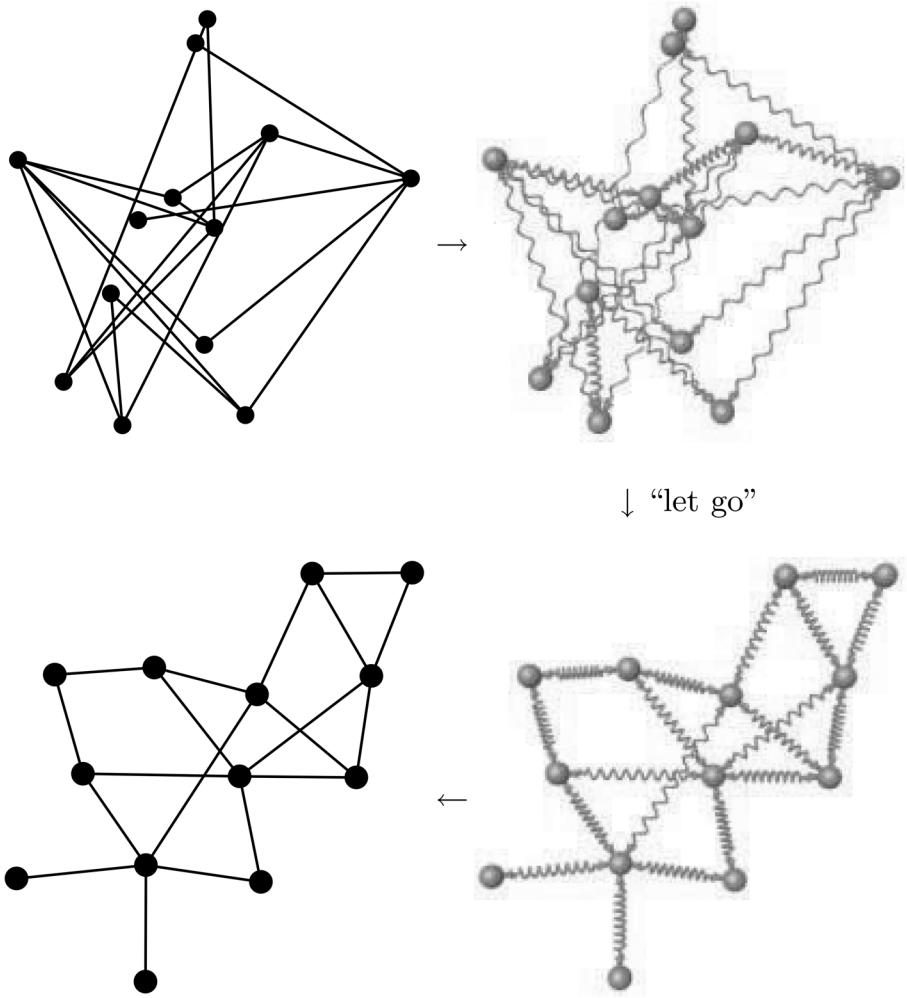


Fig. 4.1. The spring analogy.

spring embedder, implements the analogy described in the previous section. It is defined using repelling forces

$$f_{\mathsf{rep}}(p_u, p_v) = \frac{c_\varrho}{\|p_v - p_u\|^2} \cdot \overrightarrow{p_u p_v}$$

between every pair of non-adjacent vertices $u, v \in V$, where c_ϱ is a repulsion constant. Complementary spring forces between adjacent vertices $u, v \in V$ shall keep these sufficiently apart, yet close to each other. However, instead of more realistic forces according to Hooke's law, (imaginary) logarithmic springs which exert weaker forces on far apart vertices are employed. They yield forces

$$f_{\mathsf{spring}}(p_u, p_v) = c_\sigma \cdot \log \frac{\|p_u - p_v\|}{l} \cdot \overrightarrow{p_v p_u},$$

so that the direction depends on whether the actual distance is less or greater than a natural length l of the spring. Constant c_σ is a parameter controlling the strength of the spring. Figure 4.2 gives a qualitative impression of the forces a vertex u exerts on vertex v, depending on the distance between the

two. The solid line shows the force in case u is adjacent to v (f_{spring}), while the dotted line indicates the force in case u is not adjacent to v (f_{rep}). Positive values signify a force dragging v towards u, whereas negative values signify a force repelling v from u.

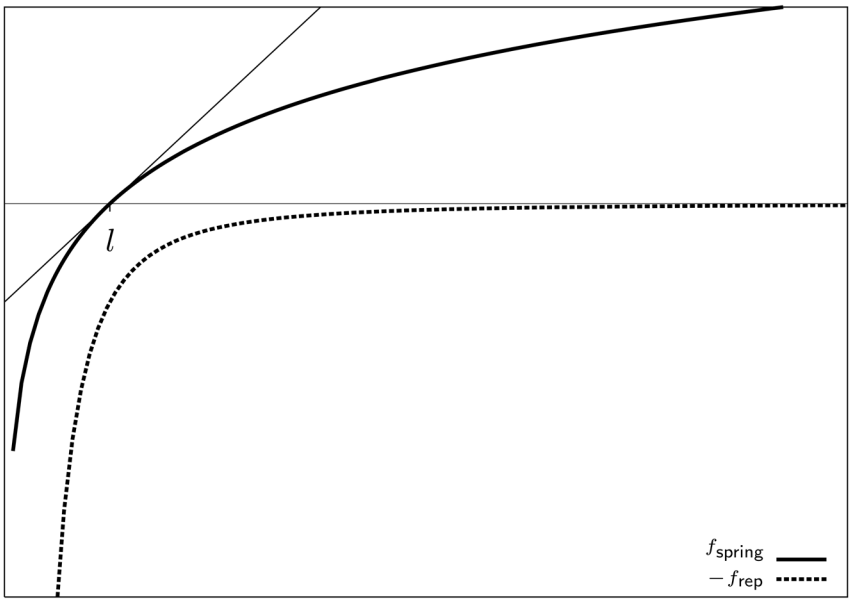


Fig. 4.2. Magnitude of spring embedder forces.

Next we address the question of how to obtain an equilibrium configuration. Vertex positions not corresponding to a system at equilibrium imply positive internal stress. To relax a stressed system, vertices are iteratively moved, at time t, according to a *net force vector* $F_v(t)$, which is the sum of all repulsion and spring forces acting on v. After computing $F_v(t)$ for all $v \in V$, each vertex is moved a constant δ times this vector. This constant is used to prevent excessive movement due to synchronous update. By iteratively computing the forces on all vertices and updating positions accordingly, the system approaches a stable state, in which no local improvement is possible. See Algorithm 6 for a concise description.

Algorithm 6: Spring embedder

Input: connected undirected graph $G = (V, E)$
initial placement $p = (p_v)_{v \in V}$

Output: placement p with low internal stress

for $t \leftarrow 1$ ***to*** ITERATIONS **do**
  for $v \in V$ **do**
    $F_v(t) \leftarrow \sum_{u\,:\,\{u,v\} \notin E} f_{\mathsf{rep}}(p_u, p_v) + \sum_{u\,:\,\{u,v\} \in E} f_{\mathsf{spring}}(p_u, p_v)$
  for $v \in V$ **do** $p_v \leftarrow p_v + \delta \cdot F_v(t)$